

Home

Library

Profile

Stories

Stats

Following

Dev Genius

Python in Plain En...

Generative AI

Realworld AI Use C...

The Useful Tech

Level Up Coding

AI Mind

Deep concept

Top Python Librari...

The Mac Alchemist

More

Member-only story

The Local AI Stack for Apple Silicon, Now With Superpowers.

Using LLM's Locally and Privately on Your Mac. Things That You Might Have Missed.



Marco Kotrotsos

Follow

9 min read · May 8, 2026



391



7



2



The important thing that changed: Ollama 0.19 (March 30, 2026) replaced its inference engine with MLX on Apple Silicon. The Ollama benchmarks on an M5 Max running Qwen3.5-35B-A3B showed prefill going from 1,154 to 1,810 tokens per second (+57%) and decode going

from 58 to 112 tokens per second (+93%). This is not a minor optimization. It is a 2x improvement on the most-used local LLM runtime.

The second: Apple's Foundation Models framework, released with macOS 26 / iOS 26 in 2025, matured through Q1 and Q2 2026 into something a Swift app can actually depend on. Guided generation through the `@Generable` macro produces type-safe structured outputs. Tool calling is built in. Multi-turn sessions are stateful. The model is 3B parameters but optimized heavily for the kinds of tasks most apps actually do (summarization, classification, structured extraction). And it costs nothing to call.

The third: macMLX shipped on April 18 as a SwiftUI-native LLM runtime with an OpenAI-compatible API. That last detail is not glamorous but it is what makes adoption tractable. Any app that already speaks OpenAI's API can swap to macMLX with a config change.

The fourth: WhisperKit (Argmax) and FluidAudio (Parakeet) now run on the Neural Engine via CoreML, and the speedups versus MLX-based transcription are dramatic. FluidAudio averages 0.19 seconds for the large model on real audio. mlx-whisper averages 1.02 seconds on the same workload. Whisper-large-v3 turbo via WhisperKit transcribes an hour of audio in roughly 90 seconds.

The combined result: the stack you would have built six months ago using Deepgram + Anthropic + a vector database can now run entirely on the user's machine, faster, for free, with unmatched privacy.

Hardware sizing: the actual recommendations

Most people overestimate what they need and underestimate what their existing hardware can do. Real recommendations by chip and memory:

M1 / M1 Pro, 8–16GB: Apple Foundation Models for everything reasonable. You have a 3B parameter on-device model already optimized for your hardware. Use it. For larger work, a 4-bit quantized 7–8B model fits but will be slow. Whisper-base or small via WhisperKit for transcription.

M2 / M2 Pro, 16–32GB: Add Qwen 3 8B at Q4 quantization (~5GB resident memory) for heavier reasoning. Keep Foundation Models for fast structured outputs. WhisperKit large-v3 turbo for transcription. This is where the hybrid stack becomes practical.

M3 Pro / M3 Max, 18–128GB: The sweet spot for solo developers. Run Qwen 3 8B for general work, Phi-4 14B (Q4, ~9GB) when you need sharper reasoning, Apple Foundation Models for structured outputs. Multiple models can stay resident simultaneously.

M4 Pro / M4 Max, 24–128GB: 30B-class models become comfortable. Llama 4 Scout (the smaller variant) at Q4 fits. DeepSeek V3-Distill-32B if you want code-heavy work. The decode speeds are now in the 60–90 tok/s range for 30B models.

M5 / M5 Max, 32–128GB: 70B-class models are practical. Qwen3.5–35B-A3B is the current MLX showcase model, hitting 112 tok/s decode after the Ollama 0.19 update. The M5's GPU Neural Accelerators specifically target LLM inference. If you have a 128GB M5 Max, you can run 70B models locally with cloud-comparable quality.

The pattern across the range: every chip from M1 forward can run something useful. The question is what model size your hardware supports comfortably while still leaving headroom for the rest of the OS to be responsive.

The four runtime choices, ranked

There are four runtimes worth knowing about. Each has a clear use case.

Apple Foundation Models (Swift framework, system-integrated)

Use this when: you are building a native macOS or iOS app and the workload fits the 3B model's strengths (summarization, classification, structured extraction, conversational tasks). The `@Generable` macro makes structured output trivial. Tool calling is built in. Zero installation friction because the model is already on every Apple Intelligence-capable device.

Skip this when: you need stronger reasoning than a 3B model can deliver, or you need a runtime that works from Python or other non-

Swift environments.

Ollama 0.19+ (MLX backend) (HTTP API, command line)

Use this when: you want a simple way to serve any of the 1000+ models in the Ollama library through a familiar REST API. The MLX backend now matches or beats most other Apple Silicon options. Wide model support (Llama, Qwen, Mistral, Phi, Gemma, DeepSeek, Mixtral, etc.). Easy to swap models with `ollama pull`.

Skip this when: you need the absolute lowest TTFT (time to first token), or you want to embed inference inside a Swift app without running a daemon.

MLX direct (Python and Swift bindings)

Use this when: you want maximum speed and tightest integration. MLX outperforms llama.cpp by 20–30% on Apple Silicon. The MLX-LM Python package gives you direct access. The Swift bindings work natively in your app. macMLX wraps it with a UI and an OpenAI-compatible API.

Skip this when: ease of use matters more than the last 20% of performance.

LM Studio (GUI app, MLX support)

Use this when: you want a desktop app to manage models, run conversations, and serve an API without command line work. Good for teams introducing local AI to non-developers. Supports MLX runtime selection.

Skip this when: you are deploying inside another app or want a programmable workflow.

Honorable mention: Rapid-MLX, an open-source MLX runtime claiming 4.2x speedup over Ollama with 0.08s cached TTFT, full tool calling, and 17 tool parsers. New (April 2026) but worth watching. Drop-in OpenAI replacement.

Model recommendations by use case

The model landscape moves fast but as of May 2026, these are the right picks for each use case:

Avoid: Llama 3.x base models (superseded), older Mistral 7B (Qwen 3 8B is better at the same size), models without active maintenance.

Speech-to-text: the sub-stack people get wrong

If you need transcription, the right answer is no longer “run Whisper through Python.”

WhisperKit (Argmax) is the production choice. Whisper variants compiled to CoreML, runs on the Neural Engine, supports streaming for real-time transcription. Whisper-large-v3 turbo via WhisperKit transcribes an hour of audio in roughly 90 seconds and works in the

foreground while you keep using your machine.

FluidAudio is the dark horse. NVIDIA's Parakeet model compiled to CoreML. Averages 0.19 seconds per inference for the large model, faster than WhisperKit on most workloads. Open source, Swift SDK. Worth using if Parakeet's accuracy holds for your domain (it is strong on English, lighter coverage on other languages).

mlx-whisper is the slowest of the three but works well in Python pipelines. Use it for batch transcription if you are already in a Python workflow.

whisper.cpp still works but is no longer the right default on Apple Silicon. The Neural Engine path through WhisperKit / FluidAudio is consistently faster.

Skip: Cloud transcription APIs for any latency-sensitive workload. Even Deepgram round-trips are slower than WhisperKit on a recent Mac for short audio.

The hybrid pattern most teams should ship

If you are building a product that uses AI for a meaningful share of its workload, the right architecture in May 2026 is a three-tier hybrid:

Tier 1: Always-on, low-latency
→ Apple Foundation Models (3B, native, free)
Use for: continuous classification, routing, structured field extraction, simple summarization

Tier 2: Heavy lift on demand
→ Qwen 3 8B via Ollama-MLX or MLX direct
Use for: detailed analysis, multi-step reasoning, longer-form generation

Tier 3: Cloud burst (optional)
→ Claude Opus 4.7 or GPT-5.5 via API
Use for: highest-stakes reasoning, only when user

```
| explicitly opts in |
```

For audio:

WhisperKit (real-time) or FluidAudio (faster batch)

The pattern has three properties that matter:

It works offline. Tier 1 and Tier 2 require no network. Your app does not break on a plane, in a hospital, in a courthouse, or when an AI provider has an outage.

It is privacy-respecting by default. Tier 3 is opt-in and explicit. The default UX never sends user data anywhere.

It is cost-stable. Tier 1 and Tier 2 are free at runtime. Tier 3 is the only cost line and is bounded by user opt-in.

This is the architecture I would build for any new AI product on Apple Silicon today. It is also the right shape for retrofitting an existing cloud-first product to a hybrid model.

What to skip

Three things people are still doing in May 2026 that they should stop:

Running everything through Python with mlx-lm. Python is fine for prototyping. For production, Swift or a native runtime gives you better integration, lower memory overhead, and access to the Neural Engine paths.

Quantizing aggressively to fit smaller hardware. Q2 or Q3 quantization saves memory but degrades quality unevenly across tasks. If your model does not fit at Q4, downgrade to a smaller model rather than quantizing harder. A Q4 8B beats a Q2 14B on most workloads.

Treating local models as cloud-quality. The 3B Foundation Models, the 8B Qwen, even the 35B Qwen3.5 are not Claude Opus 4.7. They are appropriate for the workloads they are appropriate for. Build the hybrid and route based on task complexity, not on principle.

The exact stack I would set up today

If you are starting fresh on an M3 Pro or newer Mac, this is the stack I would install:

1. **Ollama 0.19+** (`brew install ollama`). Pull Qwen 3 8B (`ollama pull qwen3:8b`) and Gemma 4 E2B (`ollama pull gemma4:e2b`).
2. **Apple Foundation Models.** Already on your system if you are running macOS 26. No install needed, just use the Swift framework.
3. **WhisperKit** (`brew install whisperkit-cli` or via Swift Package Manager). Pull the large-v3 turbo model.
4. **macMLX** for ad-hoc model exploration with a UI.
5. **Optional: Rapid-MLX** if you want the experimental fastest runtime with full tool calling.

For development, configure your IDE to point at the local Ollama endpoint (`http://localhost:11434/v1`) for any code that calls "OpenAI." Most agent frameworks (Codex, Cursor, Aider) accept this swap with a single env variable.

Total disk: ~15GB for the model files. Total RAM at idle: ~6GB resident with one model loaded. Total cost: zero per call, forever.

Why this matters strategically

Local AI on Apple Silicon is no longer a privacy story or a cost story alone. It is a deployment story. Products that ship with a tier of local inference have characteristics that cloud-only products structurally

cannot match: offline operation, predictable latency, zero per-call cost, true data residency, and resilience to provider outages.

For consumer products, this differentiation is becoming meaningful. For regulated industries (healthcare, legal, finance), it is becoming required. For developer tools, it is becoming expected.

The tooling caught up in the last 5 weeks. The hardware has been ready for a year. The question is no longer whether to run locally. It is whether you have the right hybrid architecture for your product and audience.

Set up the stack. Ship the hybrid. Stop sending data to APIs you do not need.

Marco Kotrotsos, specializing in practical AI implementation for organizations ready to close the gap between AI hype and AI value. With 30 years of IT experience now focused purely on AI deployment, he works hands-on with companies to turn AI potential into measurable business outcomes.

This article is published in Autocomplete, a Medium publication about real-world AI for practitioners and decision-makers. We're always looking for writers. If you're building with AI and have something worth sharing, reach out.

My free Substack newsletter, also called Autocomplete, can be found here: <https://acdigest.substack.com>.

Media

Artificial Intelligence

Programming

Software Development



Written by Marco Kotrotsos

3.5K followers · 412 following

Follow

Tech person. I write about technology, Generative AI, design and development.
Deeper AND broader at acdigest.substack.com. Also: <https://gloss.run>
